

## Laboratory session 3: CWS1

### Objectives

- Separate the code into two layers: user interface and business logic (which in this practice will include the business logic and persistence layers).

### Required material

In the Virtual Campus of the subject you can find:

- A compressed file `cws0.zip`. It contains a Maven project with modules `cws0`, `cws_util`, `cws_service_tests`, and several `cws_client`.
- A folder containing the HSQLDB database `cws_database`.
- A module `cws_service_interfaces` with the code of the service interfaces that must be used whenever required. These interfaces include the Data Transfer Objects (DTOs) used to exchange information between layers.

The Java version to be used will be the one installed in the laboratory computers.

We will work with the database in file mode, so it is not necessary to start it as a service. This means that when you close the application, changes made to the database will not be preserved. If you want to modify this behavior, simply update the URL parameter in the *configuration.properties* file.

**IMPORTANT: Unless explicitly stated, renaming classes, packages, methods, etc. is prohibited. This applies to both this session and to future sessions related to the CarWorkShop. It is also not allowed to move classes to a different package than the one declared in their package statement or to modify the class hierarchy implicit in the different import statements of the provided code.**

Create a workspace and import the Maven project. Starting from this initial code, you must implement the exercises. We will focus on the mechanic management use case, whose actor is the manager.

Then, run the mechanics management application, starting from the `MainMenu` class in the `manager` package.

### Exercise 1: Applying the layering pattern

Separate the code related to user interaction from the rest of the code, initially considering the mechanics management use case. Create a package structure to contain the application code, organizing it into layers according to the Layer pattern.

The `uo.ri.cws.application` package will now contain two subpackages:

- **ui**. The `ui` package that will contain classes that implement user interaction is kept.
- **service**. It will contain classes that implement both business logic and database access.

#### 1.1 Organizing the service layer

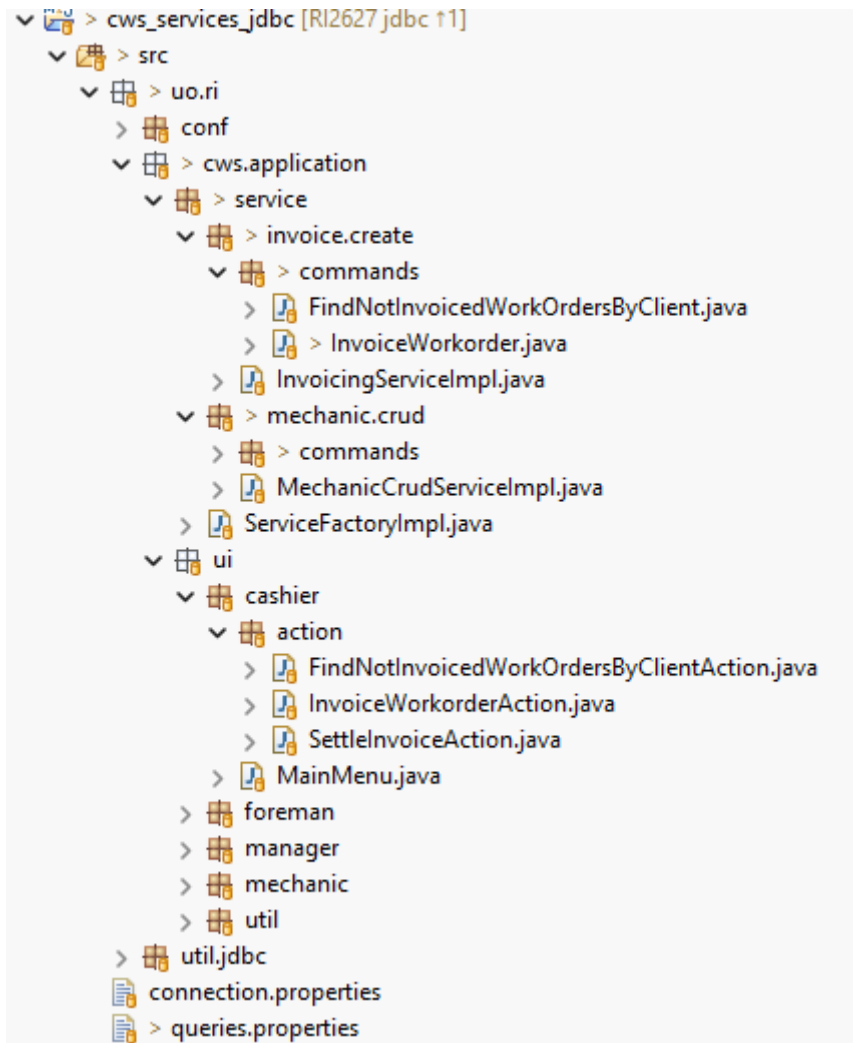
The **service layer** is organized by domain entities to increase cohesion.



- Create one package for each domain entity in the domain (initially, mechanic and invoice). Each package will handle the use case related to that entity.
- Each package will include one or more subpackages containing the classes that implement the operations of the use case.

**Style rule:** For use cases that only involve create, recover, update or delete operations, such as mechanics management, this subpackage will be named **crud**.

- Package crud will contain the implementation of the service interface. The interface is provided in the module cws\_service\_interfaces together with the DTOs (Data Transfer Objects) used to encapsulate input and output data exchanged between the service and its clients.
- Inside it, a package named **commands** will contain the classes implementing business. These classes must generally satisfy the following requirements:
  - \* Their names will match those of the classes in the ui package, removing the Action suffix.
  - \* They will have a constructor **with no parameters or with a single parameter**, which could be:
    - A DTO with the data to be added or updated.
    - A String with the ID of the object to be deleted.
    - Depending on the search criteria, find operations may or may not have arguments.
- **IMPORTANT:** The constructor must validate parameters and throw IllegalArgumentException when required. Unless otherwise specified, empty or null parameters are considered invalid. A utility class is provided in project cws\_util for this purpose (assertion.ArgumentChecks.java).
- \* A single public method **executes**, without arguments, that **will implement the business rules**, and whose return value will be:
  - For add operations: A complete DTO, including the identifier of the created object.
  - For delete or update operations: Void.
  - For retrieval operations: A DTO or a collection of DTOs, depending on the method.
- At the end of this practice, the package organization will be as shown in the figure.



## 1.2 Implementation of the service layer

- **Extract from classes `ui.manager.action.***`**, all code not directly related to user interaction or simple data validations (NIF length, NIF format, email format, etc.).
- Move business logic or data access code into **new classes located in package `service.mechanic.crud.commands`**.
- **UI classes will now invoke** the necessary classes in **the business layer**, passing the required data (DTO, String...) as parameter.
- Besides implementing the business logic required for an action, the **`execute()`** methods of the new classes should also validate business rules such as adding a repeated mechanic or deleting one that doesn't exist. For the moment, ignore them.
- A utility class `uo.ri.util.assertion.BusinessChecks` is provided to perform checks and throw `BusinessException` (package `uo.ri.util.exception`).
- **Remember to implement validations later.**

### 1.3 Implementing the UI Layer

- To print the mechanics (or other entities), use the provided **Printer** class located in package `ui.util`.

### 1.4 Functional testing

Start the database and perform the following **tests**

- List the mechanics.
- Retrieve one mechanic.
- Create a new mechanic.
- Modify the mechanic created previously.
- Delete that mechanic.
- Try to modify a mechanic whose id does not exist.
- Try to delete a mechanic whose id does not exist.

## 2 Exercise 2: Applying the Facade Pattern to decouple UI and Service Layers

To **remove dependencies** between the user interface layer and the business layer, the Facade pattern is applied, providing each subsystem (mechanical management or other) with an **interface (facade)** that acts as a **single entry point**. Clients (**UI layer or other clients**) invoke methods of the facade; they do not directly access subsystem objects (decreasing coupling). Keep these indications in mind when introducing facades into your project:

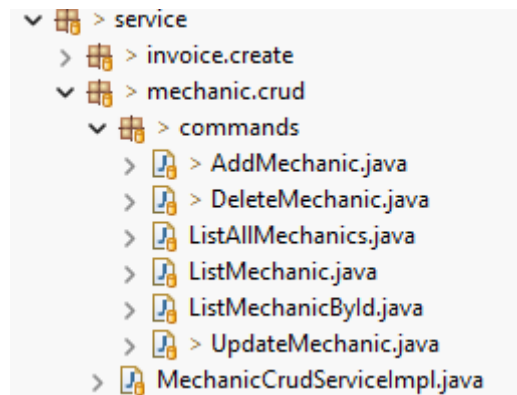
- Facades (service interfaces), provided in a separate module, must be modified (unless explicitly required).
- For the mechanics management use case, the interface is **MechanicCrudService**.

### 2.1 Service layer organization

- The **declaration** of the interfaces is located in `cws_service_interfaces` module. It is necessary to implement the Java interfaces in the implementation package `service.mechanic.crud` or `service.invoice.create`, depending on the use case.
- Each method will be responsible for invoking the corresponding implementation class (`AddMechanic`, `DeleteMechanic`, etc.). - Style guideline: The name of the Java class implementing the facade must always be the same as the interface name, with the suffix `Impl` appended.

**Style guideline:** The name of the Java class implementing the facade must always be the same as the interface name, appending the suffix **Impl**.

- Modify the `ui (***)Action` classes so that they invoke methods of the facade (`uo.ri.cws.application.service.mechanic.crud.MechanicCrudServiceImpl`), rather than service implementation objects (`AddMechanic`, `DeleteMechanic`, ...).

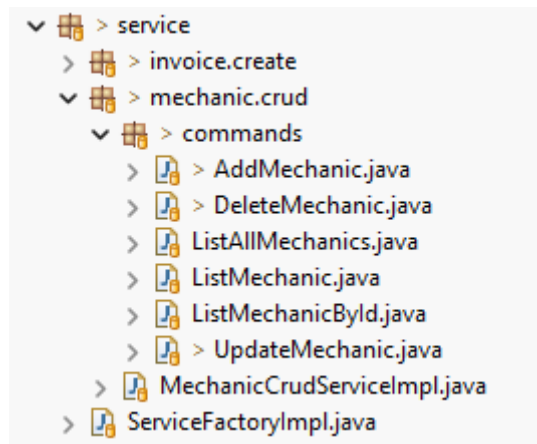


**Style rule:** If it is necessary to create new business classes, the name will be exactly the same as the name of the interface method, except for the initial capitalization. For example, a findMechanicByAge method would result in a FindMechanicByAge class.

### 3 Exercise 3: Apply the Simple Factory pattern to decouple the UI and service layers

The **Simple Factory pattern** (<https://javaee.com/factory-design-patterns-simple-factory-factory-method-static-factory-method-and-abstract-factory>) allows facade instances to be obtained without knowing how they are created (it decreases coupling). A separate class encapsulates object creation.

- The interface **ServiceFactory** is **provided**. It offers methods to obtain each of the previously defined facades (forMechanicCrudService(), forCreateInvoiceService(), etc.).
- It will act as a factory for facades allowing clients to obtain instances of service facades without knowing implementation details (thereby reducing coupling).
- The interface must be implemented by a **ServiceFactoryImpl** object in package `uo.ri.cws.service`.
- To keep all the factories centralized (the service factory and others that will appear later), a **Factories class** (another factory) is **provided with static objects that refer to the different factories that may arise**.
- **The UI will use the Factories class to obtain the appropriate factory (the service factory in this case). Once obtained, the service factory will provide the facade instance required by the client.**
  - \* Modify UI classes to remove dependencies on implementation classes in the business layer (facades, commands...).
  - \* There should be no references to implementation classes (`MechanicServiceImpl`), only to interfaces (`MechanicService`).



#### 4 Exercise 4: Validation of Business Rules.

Review classes in crud packages and verify that they perform validation of business rules such as preventing the addition of a duplicate mechanic or the deletion of a mechanic that does not exist. Under no circumstances should the application throw exceptions directly to the user.

#### 5 Exercise 5: Checking that the solution passes the tests.

Run the tests with your solution. Configure the pom.xml of the test project, if necessary.

#### 6 Exercise 6: New use cases. Billing Completed Work Orders

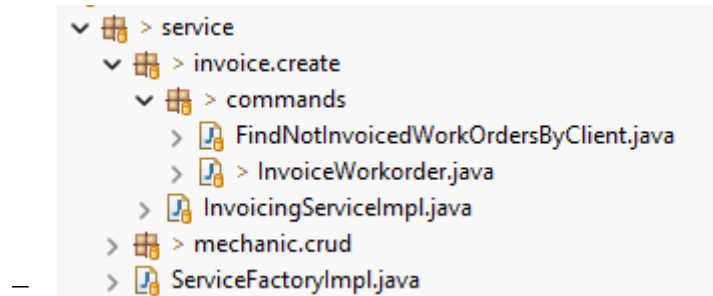
Starting from the solution to the previous exercises, implement the new use case **Invoice workorders**.

This use case is already fully implemented in the class `InvoiceWorkorderAction` in package `uo.ri.cws.application.ui.cashier.action`.

Refactor the code and reapply the previously introduced patterns to remove from class `InvoiceWorkorderAction` any code unrelated to the user interface, moving it to the appropriate classes in the business layer.

It will be organized around the entity **Invoice**:

- The implementation package for this use case will be **uo.ri.cws.application.service.invoice**.
- The interface `InvoicingService` is **provided** together with the **data transfer object** `InvoiceDto`.
- A new package **uo.ri.cws.application.service.invoice.create** will be created to implement the invoice creation use case.
- The package **uo.ri.cws.application.service.invoice.create.commands** will contain the class `CreateInvoice` that will implement the actions required to invoice workorders.



Repeat the exercise once again with the use case **Find not invoiced workorders by client**. The implementation of this use case is currently located in **ui.cashier.action.FindNotInvoicedWorkOrdersByClientAction**.

Refactor the code and reapply the previously introduced patterns, moving all logic unrelated to the user interface from the action class to the appropriate classes in the business layer.